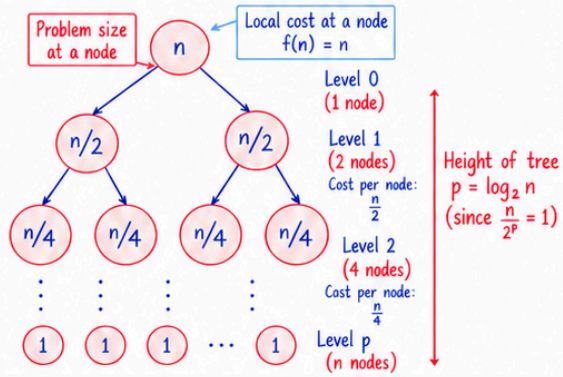
1 Recursion Tree Method

Represent the recursive calls as a tree. Calculate the cost at each level and sum up the costs of all levels to get the total time complexity.

Example: $T(n) = 2T\left(\frac{n}{2}\right) + n \;,\; T(1) = 1$



Level i	Number of nodes	Problem size per node	Cost per node $f(n/2^i)$	Total cost at level i
0	1	n	n	n
1	2	n/2	n/2	n
2	4	n/4	n/4	n
⋮	⋮	$n/2^i$	$n/2^i$	n
p	n	1	T(1) = 1	n

- Number of nodes at level i : 2^i
- Problem size at level i : $\frac{n}{2^i}$
- Cost per node at level i : $f\left(\frac{n}{2^i}\right) = \frac{n}{2^i}$
- Total cost at level i : $2^i \cdot \left(\frac{n}{2^i}\right) = n$
- Height of tree : $p = \log_2 n$
- Total internal cost : $n \cdot \log_2 n$
- Leaf cost : $n \cdot T(1) = n$
- Overall : $T(n) = n \log_2 n + n = \Theta(n \log n)$

2 Iteration Method (Repeated Substitution)

Repeatedly expand the recurrence relation until it reaches the base case. Then find the pattern and sum the terms.

Example: $T(n) = 2T\left(\frac{n}{2}\right) + n \;,\; T(1) = 1$

Expand a few times:

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n \\ &= 2\left(2T\left(\frac{n}{4}\right) + \frac{n}{2}\right) + n \\ &= 4T\left(\frac{n}{4}\right) + n + n \\ &= 4T\left(\frac{n}{4}\right) + 2n \\ &= 4\left(2T\left(\frac{n}{8}\right) + \frac{n}{4}\right) + 2n \\ &= 8T\left(\frac{n}{8}\right) + n + 2n \\ &= 8T\left(\frac{n}{8}\right) + 3n \\ &\vdots \\ &= 2^k T\left(\frac{n}{2^k}\right) + kn \end{aligned}$$

Pattern: After k expansions: $T(n) = 2^k T\left(\frac{n}{2^k}\right) + kn$

Base case: $\frac{n}{2^k} = 1 \Rightarrow k = \log_2 n$

Substitute: $T(n) = 2^{\log_2 n} T(1) + n \log_2 n$

$= n \cdot 1 + n \log_2 n$

$= n \log_2 n + n$

$\therefore T(n) = \Theta(n \log n)$

3 Substitution Method (Induction Method)

Guess a bound for T(n) and prove it using mathematical induction.

Example: $T(n) = 2T\left(\frac{n}{2}\right) + n \;,\; T(1) = 1$

Claim: $T(n) \le c\, n \log n$ for some constant $c > 0$.

Induction hypothesis:

Assume $T\left(\frac{n}{2}\right) \le c\left(\frac{n}{2}\right)\log\left(\frac{n}{2}\right)$.

Induction step:

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n \\ &\le 2\left[c\left(\frac{n}{2}\right)\log\left(\frac{n}{2}\right)\right] + n \\ &= cn(\log n - 1) + n \\ &= cn\log n - cn + n \\ &\le cn\log n \quad \text{(for } c \ge 1 \text{ and sufficiently large } n\text{)} \end{aligned}$$

Base case:

Check $T(1) \le c \cdot 1 \cdot \log 1 = 0$ (or handle separately). Hence, $T(n) = O(n \log n)$. A lower bound can be shown similarly, so $T(n) = \Theta(n \log n)$.

4 Master's Theorem

The Master Theorem applies to recurrences of the form

$$T(n) = aT(n/b) + f(n)$$

where $a \ge 1$, $b > 1$ are constants and f(n) is an asymptotically positive function.

1) If $a > b^k$, then $T(n) = \Theta(n^{\log_b a})$.
2) If $a = b^k$.
   a. If $p > -1$, then $T(n) = \Theta(n^{\log_b a} \log^{p+1} n)$.
   b. If $p = -1$, then $T(n) = \Theta(n^{\log_b a} \log\log n)$.
   c. If $p < -1$, then $T(n) = \Theta(n^{\log_b a})$.
3) If $a < b^k$
   a. If $p \ge 0$, then $T(n) = \Theta(n^k \log^p n)$.
   b. If $p < 0$, then $T(n) = O(n^k)$.

Example 1: $T(n) = 2T\left(\frac{n}{2}\right) + n$

$a = 2,\; b = 2 \Rightarrow \log_2 2 = 1$

$f(n) = n = \Theta(n^{\log_2 2})$ (Case 2 with $p = 0$)

$\therefore T(n) = \Theta(n \log n)$.

Example 2: $T(n) = 4T\left(\frac{n}{2}\right) + n^2$

$a = 4,\; b = 2 \Rightarrow \log_2 4 = 2$

$f(n) = n^2 = \Theta(n^{\log_2 4})$ (Case 2 with $p = 0$)

$\therefore T(n) = \Theta(n^2 \log n)$.

Read the following scenario

A software company uses a recursive algorithm to process a dataset containing n records, where n is a power of 2. At each recursive call, the algorithm divides the dataset into two equal-sized subproblems. It processes both subproblems recursively and then performs additional work to combine their results.

The following information is available about the algorithm:

- Dividing the dataset requires negligible time.
- Processing each subproblem follows the same algorithm.
- Combining the results of a problem of size n requires $3n + 5$ basic operations.
- Processing a dataset of size 1 requires 4 basic operations.
- The two recursive calls execute sequentially, and their running times are added.

Answer all parts of the following question, showing your working clearly. Assume that the algorithm's running time is represented by $T(n)$.

1. Based on the given information, derive the recurrence relation for $T(n)$, including the appropriate base case. Explain how each term in the recurrence is obtained.

2. Using the recurrence relation derived from part 1 :

    A. Draw the recursion tree for the first three levels, clearly showing the problem size and non-recursive cost at each node.

    B. Determine the number of nodes and the total non-recursive cost at an arbitrary level i.

    C. Identify the height of the recursion tree and calculate the total cost of all internal levels.

    D. Include the cost of the leaf nodes and derive the overall asymptotic time complexity.

3. Solve the recurrence relation using the iteration method.

    A. Expand the recurrence at least three times.

    B. Identify the general pattern after k expansions.

    C. Determine the correct value of k using the base case.

    D. Obtain a closed-form expression for $T(n)$ and state its asymptotic time complexity.

4. Use the substitution method to prove that the asymptotic upper bound obtained in part (c) is correct.

    A. State an appropriate induction hypothesis.

    B. Apply the hypothesis to the recurrence relation.

    C. Show that the resulting inequality holds for sufficiently large n.

    D. Verify the base case and explain why your argument establishes the upper bound.

5. Apply the Master Theorem to the recurrence relation.

    A. Identify a, b, and $f(n)$ in the standard form $T(n) = aT(n/b) + f(n)$.

    B. Calculate $n^{\log_b a}$.

    C. Identify the applicable Master Theorem case and justify your selection.

    D. State the final asymptotic time complexity and compare it with the results obtained using the other three methods.

6. Consider the following functions:

    - $f_1(n) = n$
    - $f_2(n) = n\log_2 n$
    - $f_3(n) = n^2$

    A. Draw a single graph showing the growth rates of all three functions, with input size n on the x-axis and the function value on the y-axis. Use a suitable range of input sizes to distinguish their growth rates

    B. Identify which curve represents the asymptotic time complexity of the recursive algorithm analysed in the previous parts

    C. Explain why the exact running time $T(n) = 3n\log_2 n + 9n - 5$ and the function $f_2(n) = n\log_2 n$ have the same asymptotic growth rate, despite having different values

7. The company develops a second version that splits the dataset **unevenly**: one subproblem gets **one quarter** of the records and the other gets **three quarters**. The combined step still costs $3n + 5$, and a dataset of size 1 still costs 4. (Ignore rounding, so all sizes are exact.) Let the running time be $S(n)$.

    A. Write the recurrence for $S(n)$.

    B. Using your result for $T(n)$, and without solving the new recurrence, guess the asymptotic growth of $S(n)$. Justify your guess with a recursive tree simply.